

~\Downloads\Medicamentos.java

```
1  import java.io.FileWriter;
2  import java.io.IOException;
3  import java.io.PrintWriter;
4  import java.time.LocalDate;
5  import java.time.LocalDateTime;
6  import java.time.format.DateTimeParseException;
7  import java.time.temporal.ChronoUnit;
8  import java.util.ArrayList;
9  import java.util.InputMismatchException;
10 import java.util.Scanner;
11
12 class Medicamento {
13     private String nome;
14     private String dosagem;
15     private int quantidade;
16     private LocalDate validade;
17
18     public Medicamento(String nome, String dosagem, int quantidade, LocalDate validade) {
19         this.nome = nome;
20         this.dosagem = dosagem;
21         this.quantidade = quantidade;
22         this.validade = validade;
23     }
24
25     public String getNome() { return nome; }
26     public String getDosagem() { return dosagem; }
27     public int getQuantidade() { return quantidade; }
28     public LocalDate getValidade() { return validade; }
29
30     public void exibirRelatorio() {
31         LocalDate hoje = LocalDate.now();
32         long diasParaVencer = ChronoUnit.DAYS.between(hoje, validade);
33
34         System.out.println("--- Relatório do Medicamento ---");
35         System.out.println("Nome: " + nome);
36         System.out.println("Dosagem: " + dosagem);
37         System.out.println("Estoque: " + quantidade + " unidades");
38         System.out.println("Validade: " + validade);
39
40         if (diasParaVencer < 0) {
41             System.out.println("Status: Vencido");
42         } else if (diasParaVencer <= 60) {
43             System.out.println("Status: Próximo da validade (" + diasParaVencer + " dias
restantes)");
44         } else {
45             System.out.println("Status: Dentro do prazo de validade (" + diasParaVencer +
" dias restantes)");
46         }
47     }
48 }
49
50 class MedicamentoControlado extends Medicamento {
```

```
51     private String numeroReceita;
52     private String crmMedico;
53
54     public MedicamentoControlado(String nome, String dosagem, int quantidade, LocalDate
validade, String numeroReceita, String crmMedico) {
55         super(nome, dosagem, quantidade, validade);
56         this.numeroReceita = numeroReceita;
57         this.crmMedico = crmMedico;
58     }
59
60     @Override
61     public void exibirRelatorio() {
62         super.exibirRelatorio();
63         System.out.println("Número da Receita: " + numeroReceita);
64         System.out.println("CRM do Médico: " + crmMedico);
65     }
66 }
67
68 public class GerenciadorEstoque {
69
70     public static void registrarLogDeErro(String modulo, Exception e) {
71         System.err.println("\n[LOG INTERNO] " + LocalDateTime.now() + " | Falha em: " +
modulo);
72         System.err.println("Erro: " + e.getClass().getSimpleName() + " " +
e.getMessage());
73
74         try (PrintWriter logWriter = new PrintWriter(new FileWriter("erros_uti.log",
true))) {
75             logWriter.println "[" + LocalDateTime.now() + "] [" + modulo + "] " +
e.toString());
76         } catch (IOException ioException) {
77             System.err.println("Falha ao gravar arquivo de log: " +
ioException.getMessage());
78         }
79     }
80
81     public static void buscarMedicamento(ArrayList<Medicamento> lista, String nomeBuscado)
{
82         boolean encontrado = false;
83         for (Medicamento med : lista) {
84             if (med.getNome().equalsIgnoreCase(nomeBuscado)) {
85                 System.out.println("\nMedicamento encontrado!");
86                 med.exibirRelatorio();
87                 encontrado = true;
88                 break;
89             }
90         }
91         if (!encontrado) {
92             System.out.println("\nMedicamento " + nomeBuscado + " não foi encontrado no
estoque.");
93         }
94     }
95
96     public static void salvarEstoqueEmArquivo(ArrayList<Medicamento> estoque, String
nomeArquivo) {
```

```
97     try (PrintWriter writer = new PrintWriter(new FileWriter(nomeArquivo))) {
98         for (Medicamento med : estoque) {
99             writer.println(med.getNome() + "," + med.getDosagem() + "," +
med.getQuantidade() + "," + med.getValidade());
100         }
101         System.out.println("\nEstoque salvo com sucesso");
102     } catch (IOException e) {
103         registrarLogDeErro("salvarEstoqueEmArquivo", e);
104         System.out.println("\nErro ao salvar o estoque: " + e.getMessage());
105     }
106 }
107
108 public static void verificarInteracao(String med1, String med2) {
109     boolean interacao = (med1.equalsIgnoreCase("Aspirina") &&
med2.equalsIgnoreCase("Varfarina")) ||
110         (med1.equalsIgnoreCase("Varfarina") &&
med2.equalsIgnoreCase("Aspirina"));
111
112     if (interacao) {
113         System.out.println("\n[ALERTA CRÍTICO] Interação Medicamentosa Detectada!");
114         System.out.println("Risco de sangramento gastrointestinal grave.");
115     } else {
116         System.out.println("\nNenhuma interação grave registrada entre " + med1 + " e
" + med2 + ".");
117     }
118 }
119
120 public static void cadastrarMedicamento(ArrayList<Medicamento> estoque, Scanner
leitor) {
121     System.out.println("=== Cadastro de Medicamentos ===");
122     try {
123         System.out.print("Digite o nome do medicamento: ");
124         String nome = leitor.nextLine().trim();
125         if (nome.isEmpty()) {
126             throw new IllegalArgumentException("O nome não pode ser vazio.");
127         }
128
129         System.out.print("Digite a dosagem: ");
130         String dosagem = leitor.nextLine().trim();
131
132         System.out.print("Digite a quantidade em estoque: ");
133         int quantidade = leitor.nextInt();
134         leitor.nextLine();
135
136         if (quantidade < 0) {
137             throw new IllegalArgumentException("A quantidade não pode ser negativa.");
138         }
139
140         System.out.print("Digite a data de validade (AAAA-MM-DD): ");
141         String validadeStr = leitor.nextLine();
142         LocalDate validade = LocalDate.parse(validadeStr);
143
144         Medicamento novoMedicamento = new Medicamento(nome, dosagem, quantidade,
validade);
```

```
145         estoque.add(novoMedicamento);
146
147         System.out.println("\nMedicamento cadastrado com sucesso!");
148
149     } catch (InputMismatchException e) {
150         registrarLogDeErro("cadastrarMedicamento - Quantidade", e);
151         System.out.println("\n[ERRO DE DIGITAÇÃO] A quantidade deve ser um número
152 inteiro.");
153         leitor.nextLine();
154     } catch (DateTimeParseException e) {
155         registrarLogDeErro("cadastrarMedicamento - Formato Data", e);
156         System.out.println("\n[ERRO DE DATA] Formato inválido! Use o padrão AAAA-MM-DD
157 (ex: 2026-12-31).");
158     } catch (IllegalArgumentException e) {
159         registrarLogDeErro("cadastrarMedicamento - Validação", e);
160         System.out.println("\n[ERRO DE VALIDAÇÃO] " + e.getMessage());
161     } catch (Exception e) {
162         registrarLogDeErro("cadastrarMedicamento - Geral", e);
163         System.out.println("\n[ERRO INESPERADO] Não foi possível realizar o
164 cadastro.");
165     }
166 }
167
168 public static void main(String[] args) {
169     Scanner leitor = new Scanner(System.in);
170     ArrayList<Medicamento> estoque = new ArrayList<>();
171
172     estoque.add(new Medicamento("Amoxicilina", "500mg", 21, LocalDate.of(2026, 12,
173 31)));
174     estoque.add(new Medicamento("Ibuprofeno", "400mg", 15, LocalDate.of(2024, 6,
175 30)));
176     estoque.add(new Medicamento("Paracetamol", "750mg", 40, LocalDate.of(2025, 5,
177 10)));
178
179     boolean executando = true;
180     while (executando) {
181         try {
182             System.out.println("\n=== MENU DO SISTEMA UTI ===");
183             System.out.println("1. Exibir todo o estoque");
184             System.out.println("2. Buscar medicamento");
185             System.out.println("3. Cadastrar novo medicamento");
186             System.out.println("4. Salvar estoque em arquivo");
187             System.out.println("5. Verificar interação medicamentosa");
188             System.out.println("6. Sair");
189             System.out.print("Escolha uma opção: ");
190
191             int opcao = leitor.nextInt();
192             leitor.nextLine();
193
194             switch (opcao) {
195                 case 1:
196                     System.out.println("\n=== LISTA COMPLETA DE ESTOQUE ===");
197                     for (Medicamento med : estoque) {
198                         med.exibirRelatorio();
199                     }
200                 case 2:
201                     // Implementação da busca por medicamento
202                 case 3:
203                     // Implementação do cadastro de novo medicamento
204                 case 4:
205                     // Implementação do salvamento do estoque em arquivo
206                 case 5:
207                     // Implementação da verificação de interação medicamentosa
208                 case 6:
209                     executando = false;
210                     break;
211                 default:
212                     System.out.println("Opção inválida. Tente novamente.");
213             }
214         } catch (Exception e) {
215             registrarLogDeErro("main - Erro no menu", e);
216             System.out.println("\n[ERRO] Ocorreu um erro ao executar o menu. Tente novamente.");
217         }
218     }
219 }
```

```
193         }
194         break;
195     case 2:
196         System.out.print("Digite o nome para busca: ");
197         String nomeBusca = leitor.nextLine();
198         buscarMedicamento(estoque, nomeBusca);
199         break;
200     case 3:
201         cadastrarMedicamento(estoque, leitor);
202         break;
203     case 4:
204         salvarEstoqueEmArquivo(estoque, "estoque_medicamentos.txt");
205         break;
206     case 5:
207         System.out.print("Digite o nome do primeiro medicamento: ");
208         String med1 = leitor.nextLine();
209         System.out.print("Digite o nome do segundo medicamento: ");
210         String med2 = leitor.nextLine();
211         verificarInteracao(med1, med2);
212         break;
213     case 6:
214         executando = false;
215         System.out.println("\nEncerrando o sistema de forma segura...");
216         break;
217     default:
218         System.out.println("\nOpção inválida! Escolha um número de 1 a
6.");
219     }
220
221     } catch (InputMismatchException e) {
222         registrarLogDeErro("main - Entrada de Opção", e);
223         System.out.println("\n[ERRO] Por favor, digite apenas números no menu!");
224         leitor.nextLine();
225     } catch (Exception e) {
226         registrarLogDeErro("main - Erro Geral", e);
227         System.out.println("\n[SISTEMA MANTIDO] Ocorreu uma falha, mas a aplicação
continua operacional.");
228     }
229 }
230
231 leitor.close();
232 salvarEstoqueEmArquivo(estoque, "estoque_medicamentos.txt");
233 }
234 }
```